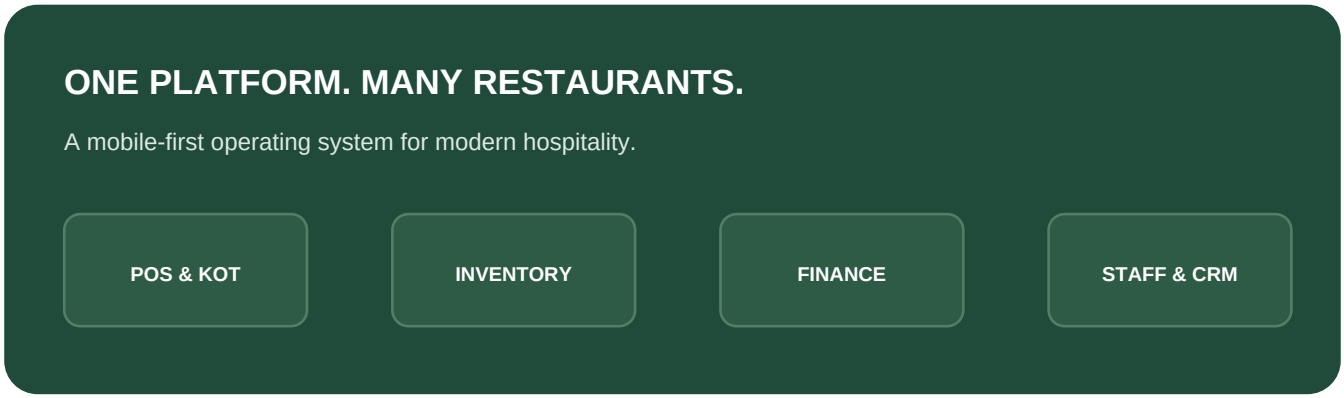
RESTAURANT MANAGEMENT SAAS

Product vision, UX direction, technical architecture, data model, security, performance, and implementation roadmap



Project principle: build from scratch, launch in one café, prove the operational workflows, then onboard additional restaurants through a secure shared SaaS platform.

Prepared: 18 October 2026 • Version 1.0 • Status: Product and implementation blueprint

This document defines the proposed product and engineering approach. It is not a claim that the application has already been built, audited, or deployed.

Document contents

01 Executive summary and product principles

02 Product scope and user roles

03 Module specifications and workflows

04 UI/UX direction and responsive behavior

05 Technical stack and rationale

06 System architecture

07 Multi-tenant model and branding

08 Database domains and data integrity

09 Security, privacy, and auditability

10 Performance and PWA strategy

11 OCR processing architecture

12 Financial reporting definitions

13 Implementation roadmap

14 Testing and acceptance criteria

15 Deployment, observability, and operations

16 Risks, assumptions, and decisions

17 Antigravity implementation workflow

18 Appendix: initial logical table inventory

01. Executive summary

The proposed product is a restaurant operating platform combining point-of-sale operations, kitchen workflows, digital menus, inventory and purchasing, financial reporting, OCR document intake, staff task management, and customer CRM in one connected system.

The product will first be deployed in one café. The software will nevertheless be architected as a multi-tenant SaaS platform from day one. Each restaurant will have its own brand identity, users, settings, branches, menu, stock, customer records, and financial data while sharing the same application and PostgreSQL database.

The product is a new build. The existing POS supplied for context is not a source of code, UI, database schema, or business logic to reuse.

Core principles

- **Operationally connected:** modules share authoritative data; an order can update KOT, payments, stock consumption, and sales reporting through controlled operations.
- **Mobile-first:** owner, staff, and QR menu workflows work well on phones; POS and kitchen screens adapt to touch displays and larger screens.
- **Secure by design:** tenant and branch access is enforced in database policies and server-side authorization.
- **Performance-aware:** route-level code splitting, small initial bundles, optimized assets, selective queries, and measured budgets.
- **Auditable:** financial and stock corrections leave traceable records; sensitive operations are not silently overwritten.
- **Phased delivery:** approve design and architecture early, implement one coherent workflow at a time, and verify before expanding scope.

Success is not defined by the number of screens. It is defined by whether staff can reliably process orders, owners can trust the financial and stock reports, and a second restaurant can be onboarded without changing application code.

Product goals

Goal	Desired outcome
Pilot launch	One café can use the platform for day-to-day workflows with measured reliability.
SaaS readiness	Additional restaurants can be provisioned through configuration and onboarding.
Tenant separation	No restaurant can access another restaurant's data, even through direct API calls.
Mobile usability	Frequent actions are easy to complete on a phone without desktop-style overflow.
Business correctness	Orders, payments, stock movements, and financial calculations reconcile.
Maintainability	Modules are typed, tested, independently understandable, and use shared UI patterns.

02. Product scope and user roles

In-scope modules

Module	Primary responsibilities
Overview dashboard	Revenue, orders, expenses, estimated profit, low-stock alerts, pending OCR reviews, staff tasks.
POS and orders	Dine-in, takeaway, delivery where enabled, cart, modifiers, discounts, tax, payment records, cancellation/refund workflow.
Tables and KOT	Table status, tickets, kitchen stations, preparation status, timers, ready/served states.
QR menu	Restaurant-branded public menu, table QR codes, availability, item customization, optional ordering.
Inventory	Ingredients and products, units, recipes, stock movements, expiry, wastage, stock counts.
Purchasing	Suppliers, purchase orders, goods received, invoice records, payment status, reorder alerts.
Finance	Sales, payments, expenses, cash reconciliation, supplier balances, cost of goods sold, profit and loss.
OCR documents	Upload, classify, extract, validate, review, approve, post, retain original.
Staff operations	Employee records, shifts, attendance, duties, hygiene checklists, document-expiry reminders.
CRM	Customer profiles, order history, feedback, loyalty and promotions with appropriate consent.
SaaS administration	Tenant onboarding, plans, entitlements, usage limits, subscription status, support tools.

User roles

Role	Expected access
Platform administrator	Manage platform operations, tenant onboarding, plans and support. No unrestricted access to tenant business data by default.
Restaurant owner	Configure restaurant, manage users, view all permitted branches, finance and reports.
Restaurant manager	Manage day-to-day operations and approved reports within assigned scope.
Cashier	Create orders, take permitted payments, issue receipts, and perform authorized corrections.
Waiter / front-of-house	Manage assigned tables and orders, send items to kitchen, and view relevant order states.
Kitchen staff	View and update kitchen tickets; no default access to financial administration.
Inventory / purchasing staff	Manage stock, purchase orders, receipts, and approved stock adjustments.
Cleaning / operations staff	View assigned duties and submit checklist evidence or completion status.
Customer	View public menu and optionally place an order or participate in loyalty features.

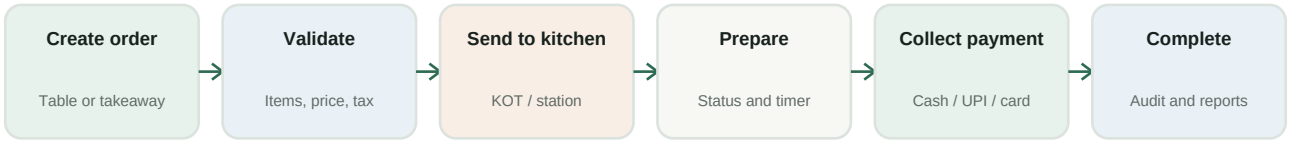
Roles should be permission bundles, not hard-coded UI assumptions. A user may hold different roles in different restaurants or branches. All sensitive actions must be authorized on the server.

03. Module specifications and workflows

3.1 POS, order lifecycle, and KOT

The POS is the operational core. It must support fast item selection and accurate order state transitions, with responsive layouts for phones, tablets, and touch displays.

- Create dine-in, takeaway, and delivery orders as enabled by the restaurant.
- Support categories, variants, modifiers, add-ons, special instructions, quantities, discounts, taxes, and service charges where configured.
- Support order hold/resume and clear cancellation/refund workflows with permission checks.
- Create kitchen tickets when items are sent to preparation; support routing by station when needed.
- Track new, accepted, preparing, ready, served/completed, cancelled, and refunded states where applicable.
- Record payment method and payment state separately from order state; support partial or split payments if required.
- Print or present customer receipts and kitchen tickets; hardware printer integrations must be verified separately.



The exact allowed state transitions must be specified. A completed order must not be casually edited; corrections should use controlled cancellation, refund, or adjustment flows.

3.2 QR menu

- Each restaurant has a public menu URL and tenant-specific branding.
- Table-specific QR codes may encode a table reference, but must not grant authorization to private data.
- Customers can browse without signing in; account creation is optional unless a feature truly requires it.
- Menu availability, variants, modifiers, allergen information, prices, and opening hours are configuration-driven.
- Public menu bundles should be particularly small; customer menu routes must not download owner dashboard, OCR, or finance code.

3.3 Inventory and purchasing

- Maintain ingredients and sellable items with units of measure and explicit conversion rules.
- Use recipe/BOM records to calculate theoretical ingredient consumption from completed or otherwise defined sale events.
- Record every stock change as a movement with source, actor, timestamp, quantity, unit, and reason.
- Track supplier, purchase order, goods receipt, supplier invoice, payment status, and stock received as related but distinct records.
- Support expiry/batch data where needed, wastage reasons, physical stock counts, low-stock thresholds, and reorder suggestions.
- Reconcile theoretical and physical stock; do not silently rewrite history when recipes or units change.

3.4 Finance and reporting

- Record sales, payments, expenses, purchases, supplier liabilities, refunds, and cash movements as separate concepts.
- Provide daily, weekly, monthly, and custom-period reports with clear filters and export options.
- Distinguish gross sales, net sales, collected payments, cash inflow, cash outflow, purchases, cost of goods sold, gross profit, and net profit.

- Show reconciliation status and identify incomplete or unapproved transactions.
- Do not equate purchases with cost of goods sold; purchased stock may remain in inventory.
- Do not label revenue minus recorded purchases as net profit. Define accounting treatment and reporting periods explicitly.

3.5 Staff and checklists

- Manage employee profiles, branch assignments, roles, shifts, attendance, and assigned tasks.
- Provide opening, kitchen, dining area, washroom, equipment, storage, and closing checklists as configurable templates.
- Capture assigned person, due time, completion time, evidence if required, and exception/corrective action.
- Track relevant health or compliance document dates and expiry reminders with strict access to sensitive documents.
- Do not expose sensitive employee health details to all staff; keep only necessary records and define retention/access policy.

3.6 CRM

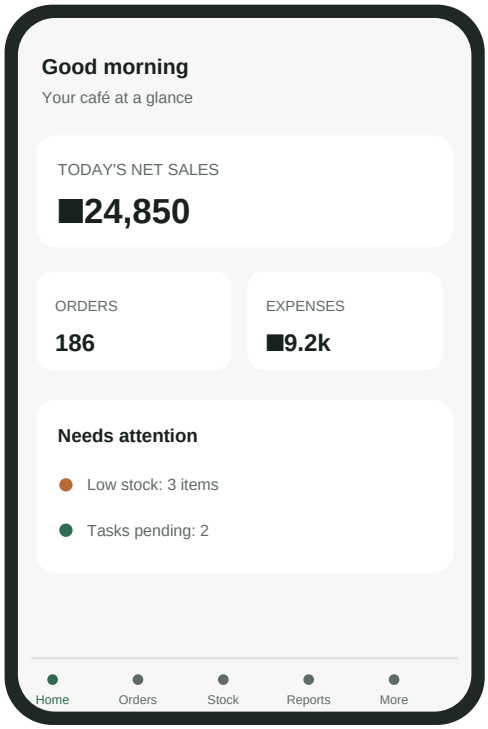
- Maintain customer profiles and order history where collection is appropriate.
- Support feedback, loyalty points, rewards, customer preferences, and complaint follow-up.
- Use explicit consent and communication preferences for marketing.
- Allow QR menu browsing without forcing account creation.

04. UI/UX direction and responsive behavior

The visual system should feel like polished commercial software: restrained, precise, and consistent. It should not rely on gradients, excessive rounded cards, oversized headings, decorative charts, or repeated template layouts to appear modern.

Visual direction

Area	Design decision
Overall aesthetic	Refined enterprise SaaS with subtle hospitality character.
Color	Neutral surfaces, one restrained tenant brand accent, semantic status colors.
Typography	One highly readable UI family and a documented type scale.
Spacing	A consistent spacing scale; avoid arbitrary per-component values.
Components	Shared, accessible controls with consistent states and interaction behavior.
Charts	Only when they answer a real operational question; lazy-load expensive chart code.
Motion	Purposeful short transitions; honor reduced-motion settings.
Branding	Tenant configuration mapped to CSS variables; do not create a separate build per restaurant.



Mobile-first layout principles

01 Prioritize decisions
Sales and exceptions appear first.

02 Keep actions reachable
Bottom navigation for frequent tasks.

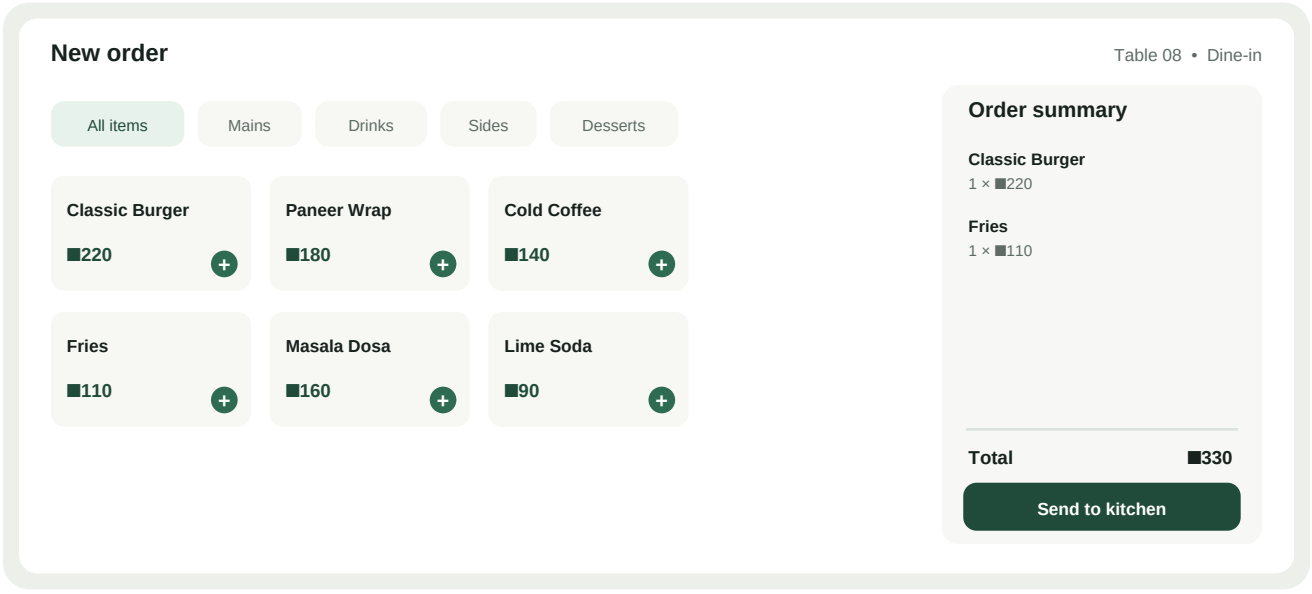
03 Use compact hierarchy
Avoid oversized cards and wasted space.

04 Design for touch
Large targets and clear task states.

05 Show real status
Loading, empty, error and offline states.

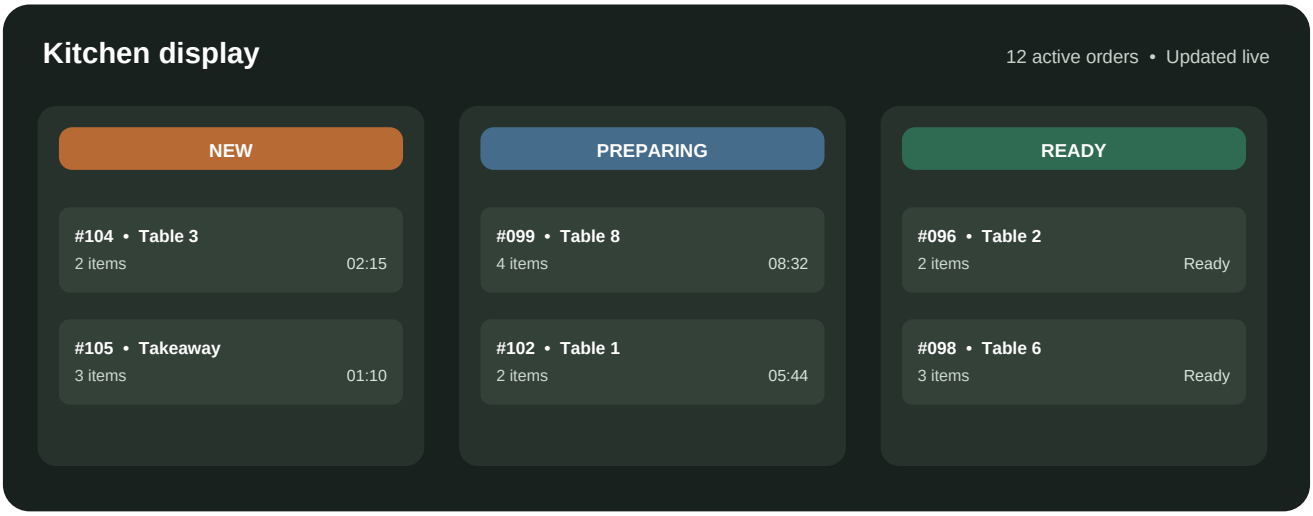
Illustrative mobile dashboard wireframe. Values are sample data for layout demonstration only.

POS screen concept



Illustrative POS wireframe: categories and item grid on one side, order summary and primary kitchen action on the other. The mobile layout must reflow these regions rather than shrink the desktop layout.

Kitchen display concept



Illustrative kitchen display: high-contrast order cards grouped by preparation state. Timers and state changes must remain readable from a working distance.

Responsive behavior by device

Device / viewport	Layout and interaction
Phone (360–430 px)	Bottom navigation for frequent tasks; one-column or compact two-column cards; full-screen complex forms; no accidental horizontal overflow.
Tablet (600–1024 px)	Touch-optimized POS; adaptable split panels; tables and forms use available width without excessive density.
Desktop (≥1024 px)	Collapsible sidebar, wider data tables, multi-panel POS, detailed reporting and administrative workflows.
Kitchen display	High-contrast full-screen layout, large order cards, clear status transitions, optional sound alerts with accessible visual alternatives.

Touch targets should generally be around 44–48 CSS pixels where practical. Test keyboard behavior, safe areas, focus indicators, reduced motion, screen-reader labels, and the on-screen keyboard.

05. Technology stack and rationale

Layer	Recommended choice	Implementation notes
Frontend	React + TypeScript + Vite	Typed SPA, fast development/build workflow, client-side operational UI.
Styling	Tailwind CSS + CSS variables	Tokens, responsive utilities, tenant theme variables.
Accessible UI	Radix UI primitives	Use selectively for dialogs, dropdowns, menus, tabs, and popovers.
Routing	React Router	Lazy-load each module/route; keep public menu separate from admin bundle.
Server state	TanStack Query	Cache, request lifecycle, invalidation, pagination, refetch control.
Local UI state	React state; Zustand if justified	Avoid turning server data into duplicated global state.
Forms	React Hook Form + Zod	Reusable schemas and clear validation; validate again server-side.
Database	PostgreSQL via Supabase	Relational constraints, indexes, transactions and RLS.
Identity	Supabase Auth	Sessions, login and recovery; membership/permissions remain application-domain data.
Storage	Supabase Storage	Restaurant logos, receipt scans, invoices, attachments with access policies.
Server functions	Supabase Edge Functions / trusted API layer	Sensitive business operations, authorization, external service integration.
Realtime	Supabase Realtime selectively	KOT/order status and other time-sensitive updates; avoid subscribing to everything.
OCR worker	Separate worker when justified	Retries, job status, extraction, confidence scores, long-running processing.
Hosting	Vercel + Supabase	CDN-backed frontend and managed database/backend services.
Testing	Vitest, React Testing Library, Playwright	Unit, integration, browser, and end-to-end workflows.

Why not over-engineer the backend?

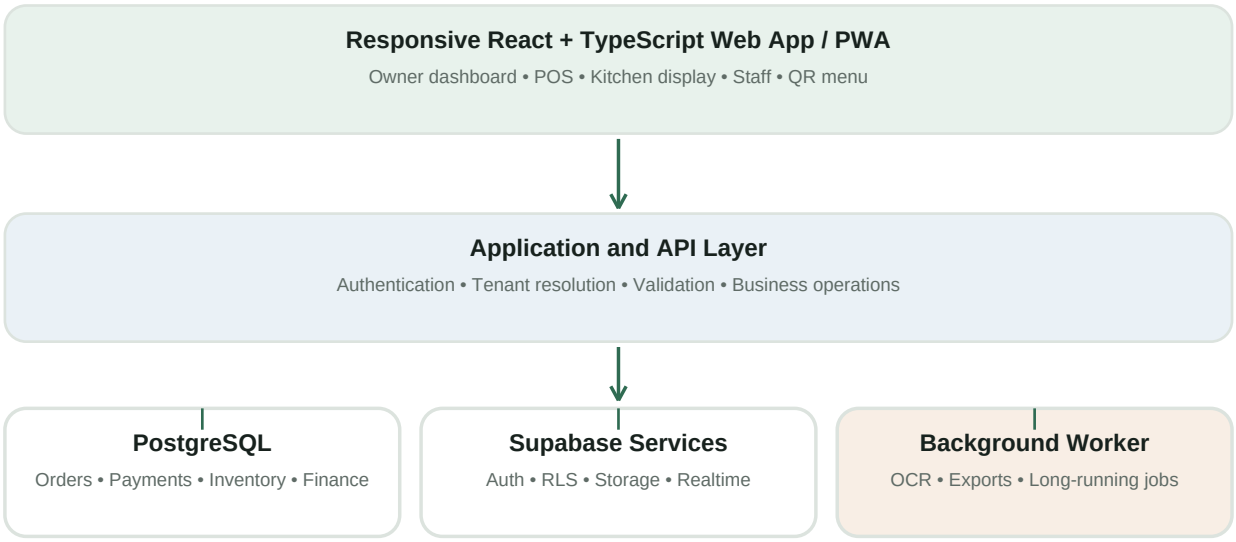
A modular monolith is the preferred starting point: one coherent application, a shared PostgreSQL database, clearly separated business modules, and server-side functions for sensitive operations. This keeps operational complexity manageable while allowing OCR or other workloads to move to separate workers later.

Public menu versus authenticated application

The public QR menu is a separate lightweight route and bundle. It should not load owner dashboards, financial charts, staff administration, or OCR tooling. The authenticated application loads module code only when the user navigates to it.

React + Vite is a suitable choice for this operational platform. A separate server-rendered public site can be considered later if public SEO becomes a major requirement.

06. System architecture



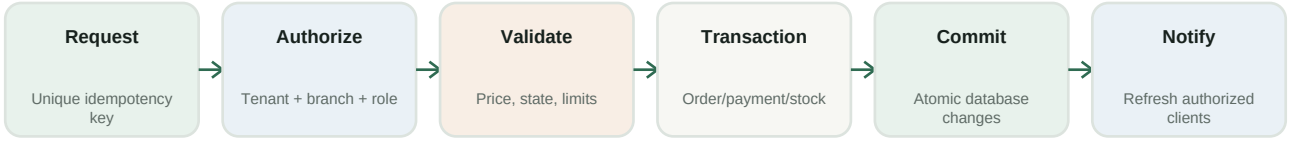
Tenant isolation is enforced by server authorization and database policies—not by UI filtering.

Architecture responsibilities

Component	Responsibilities
Frontend	Presentation, navigation, input validation for usability, cached server state, responsive interaction. Never the authority for permissions or financial calculations.
Trusted API / server operations	Resolve authenticated identity and tenant membership; validate inputs; authorize actions; orchestrate transactions and external calls.
PostgreSQL	Authoritative relational records, foreign keys, unique constraints, transactions, indexes, and RLS policies.
Supabase Auth	Identity and session lifecycle; app membership and role assignments are managed separately.
Storage	Private documents with scoped access; public assets only when intentionally public.
Realtime	Notify authorized clients of relevant state changes; do not use realtime messages as the authoritative record.
OCR worker	Process queued documents, retry safely, store extraction confidence and errors, and await approval before posting.

Key transactional workflows

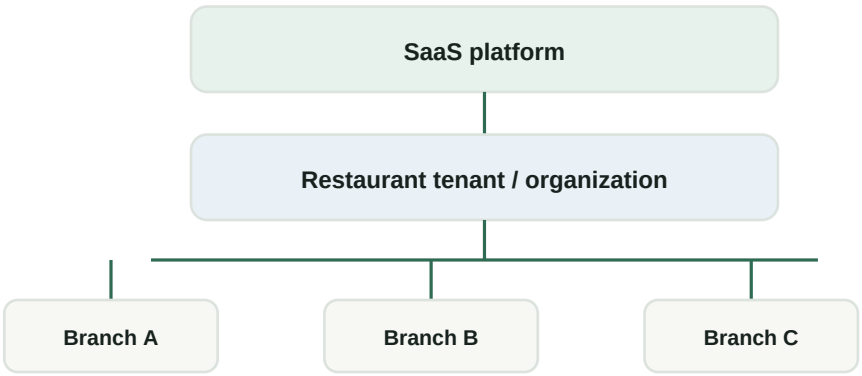
Order completion must be defined as a server-side workflow. It should validate current state and payment conditions, record the appropriate payment/order changes, and create stock/financial effects exactly once according to business rules.



If a workflow crosses systems that cannot share one database transaction, use an explicit job/outbox or retryable workflow with idempotency and reconciliation. Never assume that a sequence of independent browser requests is atomic.

07. Multi-tenant model and branding

Tenant hierarchy



One organization may operate multiple branches. Memberships connect users to organizations and define roles and scope. Branch-specific stock, orders, shifts, cash sessions, and operational records should reference the relevant branch.

Brand configuration

Configuration	Examples
Identity	Restaurant display name, slug, logo, optional favicon.
Theme	Primary/accent colors, theme preferences, logo variants.
Business settings	Timezone, currency, tax settings, receipt details, operating hours.
Ordering	Menu visibility, QR ordering settings, table mapping, service modes.
Branches	Branch name, address, operating hours, local settings, permitted staff.
Domain	Default platform URL; optional verified custom domain mapped to a tenant.

Isolation rules

- Every tenant-owned record has a tenant/organization foreign key; branch-scoped records also reference a branch.
- RLS policies verify membership and allowed scope. Client-supplied tenant IDs are not trusted as proof of access.
- Sensitive actions are checked in trusted server-side code as well as protected by database constraints/policies.
- Platform support access is explicit, least-privilege, auditable, and not equivalent to unrestricted access to every tenant's business data.
- Automated tests attempt cross-tenant reads, inserts, updates, deletes, storage access, and realtime subscription access.
- Custom domain resolution verifies ownership and maps the hostname to a tenant using trusted configuration.

A shared database does not mean shared visibility. The isolation boundary must be enforced by authorization and database policies for every tenant-owned resource.

08. Database domains and data integrity

The following is a logical domain model. Exact columns, indexes, constraints, and normalization should be finalized in reviewed migrations before implementation.

Domain	Core entities (initial logical inventory)
Identity and tenancy	organizations, branches, user_profiles, organization_memberships, branch_memberships, roles, permissions, role_permissions, subscriptions, plan_entitlements
Restaurant configuration	restaurant_settings, brand_assets, custom_domains, operating_hours, tax_settings, receipt_settings
Menu and tables	menu_categories, menu_items, item_variants, modifier_groups, modifiers, menu_item_modifiers, restaurant_tables, qr_codes
Orders and KOT	orders, order_items, order_item_modifiers, order_events, kot_tickets, kot_items, kitchen_stations, payments, refunds
Inventory	inventory_items, units_of_measure, unit_conversions, recipes, recipe_items, stock_batches, stock_movements, stock_counts, stock_count_items, wastage_records
Purchasing	suppliers, purchase_orders, purchase_order_items, goods_receipts, goods_receipt_items, supplier_invoices, supplier_payments
Finance	expenses, expense_categories, financial_transactions, cash_sessions, cash_movements, settlement_records, accounting_periods
OCR and documents	documents, document_jobs, ocr_extractions, ocr_line_items, document_reviews, document_postings
Staff operations	employees, shifts, attendance_events, task_templates, checklist_templates, checklist_runs, checklist_items, checklist_completions, compliance_documents
CRM	customers, customer_consents, feedback, loyalty_accounts, loyalty_transactions, promotions
Audit and platform	audit_events, idempotency_records, notification_jobs, background_jobs, webhook_events

Integrity requirements

- Use foreign keys for relationships; avoid joining customers or suppliers solely by display-name strings.
- Use integer minor units for money or a carefully defined PostgreSQL numeric strategy; do not rely on floating-point arithmetic for monetary totals.
- Snapshot sold item names, prices, modifiers, and tax details onto order items so future menu changes do not rewrite historical sales.
- Version recipes or retain enough historical cost data to make prior cost calculations explainable.
- Record stock movements rather than simply overwriting quantities without history.
- Use unique constraints and idempotency keys for external requests, payments, and retryable operations.
- Use explicit timezone handling: store timestamps consistently and render in the restaurant's configured timezone.
- Use soft deletion only where it makes sense; preserve financial/audit history according to retention policy.

Suggested query/index strategy

Common query paths will include tenant + branch + date, tenant + status, order ID + item rows, supplier + invoice date, and inventory item + movement date. Indexes should follow measured query patterns; adding indexes to every column increases write cost and storage.

09. Security, privacy, and auditability

Control	Requirement
Authentication	Use established session management and secure account recovery; do not implement password storage manually.
Authorization	Check organization membership, branch scope, and specific action permission server-side.
RLS	Enable and test policies on every tenant-owned table and private storage path.
Secrets	Keep service-role keys, OCR provider keys, and webhook secrets in trusted server-side environment configuration only.
Input validation	Validate untrusted inputs at runtime on the server; client validation is for usability, not security.
Audit trail	Record actor, tenant, branch, action, timestamp, target record, and relevant before/after details for critical operations.
Payment safety	Verify provider webhooks, protect against replay, use idempotency, and reconcile settlements.
Document privacy	Restrict invoice and health-document access; use signed URLs or controlled access for private files.
Rate limits	Protect public QR ordering, login, OCR uploads, and sensitive endpoints from abuse.
Backups	Define backup frequency, retention, recovery objectives, and perform restore drills.
Data lifecycle	Define retention/deletion rules for customer data, health documents, invoices, logs, and backups.
Monitoring	Track failures, unusual access, queue delays, slow queries, and transaction/reconciliation anomalies.

Privacy-sensitive employee information

Health-related documents and compliance records require strict access control and data minimization. The application should track relevant check dates and expiry reminders without making sensitive details visible to all managers or employees. Define a lawful purpose, retention policy, and access audit before storing such information.

Security release gate

The application must not onboard a second independent restaurant until cross-tenant isolation tests, role tests, storage access tests, and direct API authorization tests pass. A polished frontend is not evidence of security.

10. Performance and PWA strategy

The performance objective is to minimize the work and data required to display each screen, rather than impose an unrealistic size limit on the entire product.

Metric	Initial target / policy	How to validate
Public menu initial JS	Aim for 50–100 KB compressed	Production build report + fresh browser network capture.
Authenticated dashboard initial JS	Aim for 100–200 KB compressed	Production build report + route-specific network capture.
Other modules	Loaded on demand	Verify chunks are not downloaded before navigation.
Largest Contentful Paint	≤ 2.5 s on representative mobile conditions	Field data and repeatable lab tests.
Interaction to Next Paint	≤ 200 ms	Measure representative interactive workflows.
Cumulative Layout Shift	≤ 0.1	Measure initial load and async content behavior.
API payloads	Small, paginated, and purpose-specific	Network inspection and query logs.
Images and fonts	Responsive formats; limited font weights	Asset inspection and transfer-size measurement.

Implementation techniques

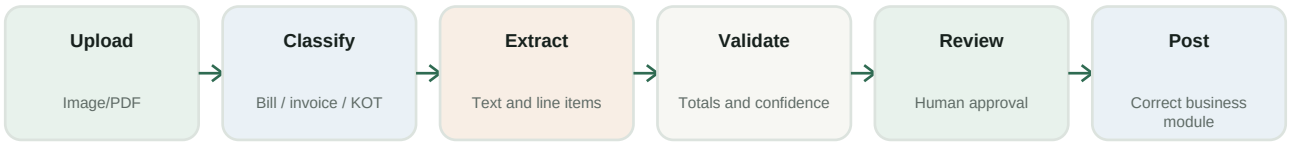
- Route-level lazy loading and dynamic imports for finance charts, OCR review, reports, and administration.
- Tree-shaking and production minification; inspect dependency and bundle analyzer output.
- Load only required fields and rows; paginate large lists and use server-side aggregation for reports.
- Use query caching and invalidation based on domain events; avoid refetching all data on every navigation.
- Optimize images with WebP/AVIF where supported, correct dimensions, thumbnails, and lazy loading.
- Use Brotli/gzip compression, long-lived caching for hashed assets, and a CDN.
- Keep the public QR menu bundle separate from authenticated admin and operational code.
- Measure cold start, repeat navigation, weak mobile network, low-end devices, and large tenant datasets.

PWA and offline policy

An installable PWA can cache versioned static assets and provide an app-like launch experience. Offline POS is a separate feature requiring durable local persistence, conflict handling, unique transaction identifiers, and reconciliation. The first release should not promise offline financial posting until those mechanisms are implemented and tested.

The 20 KB ambition should be treated as a motivation to keep each initial route small. It is not a promise that the entire application, its assets, and its API traffic will total 20 KB.

11. OCR document processing architecture



Supported document categories

Document type	Typical extraction and destination
Thermal receipt	Merchant, date, subtotal, tax, total, payment clues; expense workflow if applicable.
A4 supplier invoice	Supplier, invoice number, date, line items, quantities, unit rates, taxes, total, payment terms; purchase workflow.
Purchase bill	Supplier and purchased items; review before goods receipt, stock update, and financial posting.
KOT slip	Order/ticket number, items, quantities, notes, station clues; order reconciliation, not automatically a purchase expense.

Processing controls

- Keep the original uploaded file and record who uploaded it and when.
- Run processing asynchronously with explicit job status: queued, processing, needs review, approved, rejected, failed, or posted.
- Store extracted fields, line items, confidence scores, model/provider metadata where appropriate, and validation results.
- Recalculate line totals and compare extracted totals; flag mismatches instead of silently correcting them.
- Detect likely duplicates using tenant, supplier, invoice number, date, total, and file fingerprints as appropriate.
- Require a human review when confidence is low, totals disagree, the document type is uncertain, or posting has financial/stock impact.
- Make posting idempotent so retries cannot create duplicate expenses, purchase receipts, or stock movements.
- Protect document storage with tenant/role-scoped authorization.

Recommended rollout

Start with one common supplier-invoice format and one expense receipt workflow. Measure extraction accuracy on real, consented test documents, then expand to varied thermal receipts and KOT layouts. OCR accuracy must be reported by field and document type; a single aggregate accuracy number can hide failures in totals or tax fields.

12. Financial reporting definitions

Financial dashboards must use consistent, documented definitions. The application should distinguish sales activity, actual cash movement, inventory consumption, and profit.

Metric	Suggested definition / caution
Gross sales	Value of recorded sales before the specified discounts/adjustments, according to a documented policy.
Net sales	Gross sales adjusted for discounts, returns, and relevant sales adjustments.
Payments collected	Successful payments received; may differ from sales in a period due to timing, dues, refunds, or settlements.
Cash inflow	Cash actually received through recorded cash movements; distinct from non-cash sales.
Cash outflow	Money actually paid out; supplier invoices may be recorded before payment.
Purchases	Goods ordered/received/billed, depending on the selected report basis. Clearly label the basis.
Cost of goods sold	Cost assigned to items sold/consumed under the chosen inventory costing policy.
Gross profit	Net sales less cost of goods sold under the defined policy.
Operating expenses	Rent, salaries, utilities, maintenance, and other operating costs according to accounting policy.
Net profit	Revenue less applicable costs and expenses for the selected period; subject to completeness and accounting treatment.
Cash drawer variance	Expected cash compared with counted cash after recorded cash in/out movements.

Reporting safeguards

- Show the reporting period, branch, currency, and whether figures are provisional or finalized.
- Distinguish transaction date, invoice date, payment date, and settlement date where relevant.
- Do not treat a purchase as immediate cost of goods sold if inventory remains unused.
- Keep refunds, voids, discounts, and corrections traceable.
- Provide reconciliation indicators and drill-down from dashboard totals to source records.
- Have accounting and tax treatment reviewed for the intended jurisdiction before using the platform as a statutory accounting or tax-filing system.

13. Implementation roadmap

Phase	Scope	Deliverable / exit gate
1. Specification	Requirements, workflows, role matrix, architecture, entity model, design and performance standards.	Reviewed product specification and approved design decisions.
2. Design system	Responsive shell, tokens, navigation, component library, dashboard/POS/inventory prototypes.	Visual review passed at mobile, tablet and desktop widths.
3. SaaS foundation	Auth, organizations, branches, memberships, roles, RLS, audit, migrations, environments.	Cross-tenant and role tests pass.
4. POS / KOT / QR	Menu, tables, orders, modifiers, ticket states, payment records, QR menu.	Order-to-kitchen-to-payment workflow is consistent.
5. Inventory / purchasing	Recipes, stock movements, suppliers, purchase orders, goods receipt, wastage.	Sales and purchases update inventory correctly.
6. Finance	Expenses, cash sessions, sales reports, COGS, P&L;, supplier balances.	Totals reconcile to source transactions.
7. OCR / staff	Document pipeline, approval, staff records, attendance, hygiene checklists.	OCR posts only through validated approval flow; tasks auditable.
8. CRM / SaaS admin	Customers, loyalty, subscriptions, onboarding, entitlements.	Second tenant onboarded without code changes.
9. Pilot / launch	Real café operations, device/printer checks, load and security tests, backup/restore drill.	Operational acceptance and launch checklist complete.

Suggested first milestone

The first development milestone should not be a fully functional restaurant POS. It should be a reviewed product specification plus an approved visual prototype and a secure tenancy/data model. This reduces the risk of generating many screens on top of incorrect assumptions.

Dependency notes

- Tenant and branch isolation must be part of the initial schema and tests, not a later migration shortcut.
- POS, KOT, payment, and stock semantics must be agreed before finance reports are finalized.
- Recipe consumption depends on defined order events, unit conversions, and inventory movements.
- OCR depends on document records, private storage, review permissions, and idempotent posting.
- Subscriptions can be implemented after core tenant isolation is stable, but plan/entitlement boundaries should be designed early.

14. Testing and acceptance criteria

Test layer	What to test
Unit tests	Money rounding, tax calculations, recipe quantity conversion, status transition rules, report calculations, validation schemas.
Database tests	Foreign keys, constraints, transaction behavior, tenant/branch RLS, idempotency uniqueness, audit creation.
Integration tests	Create order, send KOT, collect payment, complete order, record stock movements, compute report totals.
Security tests	Cross-tenant reads/writes, role escalation attempts, direct API calls, private file access, public endpoint abuse.
End-to-end tests	Owner onboarding, menu configuration, POS order, kitchen completion, purchase receipt, expense approval, checklist completion.
Responsive tests	360 px, 390 px, tablet, desktop; keyboard, scrolling, touch targets, safe areas, loading states.
Performance tests	Production bundle, cold route load, API payload sizes, slow query logs, weak network, low-end devices.
Resilience tests	Retry after timeout, duplicate webhook, worker retry, failed OCR, interrupted request, backup restoration.

Minimum release acceptance criteria

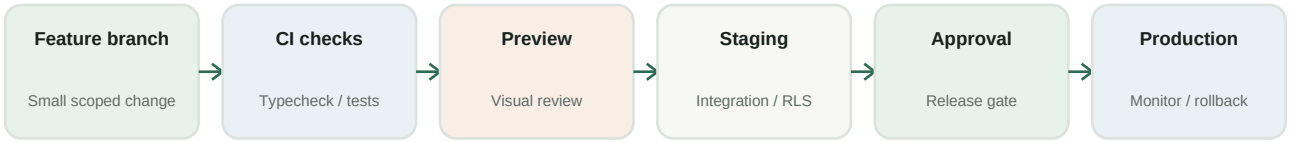
- No cross-tenant or unauthorized branch access in automated tests.
- Critical financial and stock operations are idempotent or protected against duplicate effects.
- Order, payment, refund, KOT, and stock states have documented allowed transitions.
- Financial reports reconcile to source records under documented rules.
- Core mobile flows do not require horizontal scrolling at supported widths.
- Production builds pass TypeScript checks and automated tests; bundle budgets are measured.
- Errors are observable without leaking sensitive customer, financial, or health information.
- Backups and restore procedures have been tested, not merely configured.

15. Deployment, observability, and operations

Environments

Environment	Purpose	Rules
Local development	Feature work and test data.	Local or non-production configuration; no production secrets.
Staging	Integration, QA, tenant-isolation tests, performance tests.	Representative configuration and sanitized/test data.
Production	Live café and later tenant workloads.	Controlled migrations, monitored rollout, backups, access restrictions.

Deployment workflow



Observability

- Structured server logs with correlation/request IDs, tenant ID only where safe, and redaction of sensitive fields.
- Metrics for API latency, error rate, database connection/slow-query behavior, background queue depth, OCR failures, and order workflow failures.
- Alerts for failed payment webhooks, reconciliation mismatches, stuck OCR jobs, and abnormal authorization failures.
- Audit logs for privileged actions and business corrections.
- Error tracking with source maps managed securely and without leaking secrets.
- Document a rollback process for frontend deployments and database migrations; favor additive, backward-compatible schema changes.

Operational responsibilities

Before launch, decide who handles customer support, failed payments, data export requests, incident response, retention/deletion requests, and restoration. A SaaS product is an ongoing operational service, not only a deployed codebase.

16. Risks, assumptions, and decisions

Risk / open decision	Why it matters	Recommended handling
Scope overload	Building every module at once creates inconsistent workflows and shallow testing.	Implement phased vertical workflows with explicit exit criteria.
Tenant leakage	Shared database makes policy mistakes high impact.	RLS + server authorization + automated negative tests before second tenant.
Financial ambiguity	Revenue, cash flow, purchases, COGS, and profit are different concepts.	Document accounting definitions and reconcile reports to transactions.
Offline expectations	PWA caching alone does not make POS safely offline.	Defer offline posting until local persistence and conflict reconciliation are designed.
Printer compatibility	Browser printing may not cover all Bluetooth/USB/network printer workflows.	Test target hardware early; decide whether a local print bridge or supported print path is needed.
OCR inaccuracies	Poor scans or unusual layouts can misread totals and tax.	Human review, confidence thresholds, duplicate checks, audit trail.
Performance drift	New modules can increase initial JS and API payloads.	Maintain route budgets and CI bundle checks.
Data privacy	Employee health documents and customer information require controls.	Minimize collection, restrict access, define retention and consent.
Subscription billing	Plan limits and billing status affect tenant access.	Design entitlements early; integrate billing after core operations stabilize.
Tax/accounting scope	Statutory compliance differs by jurisdiction and may change.	Have local tax/accounting requirements reviewed before claiming compliance.

Assumptions in this blueprint

- The initial implementation targets a café in India, with currency, timezone, tax and receipt settings configurable per tenant.
- Core ordering and management flows are online-first; reliable offline POS is not included by default.
- QR menu browsing is public; online ordering can be enabled by restaurant configuration.
- Payment gateway and hardware printer choices remain to be selected and tested.
- Financial reports are management reporting until statutory accounting requirements are separately defined.
- OCR provider selection will be based on document types, accuracy, cost, data handling, and latency.

17. Antigravity implementation workflow

Antigravity should be used in bounded stages. Each prompt should define scope, files/areas allowed to change, deliverables, tests, and what must not be implemented yet.

Prompt stage	Instruction focus	Expected output
Prompt 1	Product specification and architecture only.	PRD, role matrix, architecture, ERD, security model, roadmap, open decisions.
Review gate	Review the proposed schema and workflow semantics.	Correct gaps before generating code.
Prompt 2	Design system and three responsive screens only.	Tokens, navigation, dashboard, POS and inventory prototypes.
Review gate	Inspect screenshots and responsive behavior.	Approve visual quality and component patterns.
Prompt 3	Secure multi-tenant foundation.	Auth, organization/branch model, RLS, migrations, tenant-isolation tests.
Prompts 4–8	Implement one business phase at a time.	POS/KOT/QR, inventory/purchasing, finance, OCR/staff, CRM/admin.
Release gate	Security, integration, performance and recovery tests.	Evidence of acceptance criteria and measured build sizes.

Rules for every implementation prompt

- Do not implement unrelated future modules in the current phase.
- Do not invent or bypass approved schema, security, or financial rules.
- Do not expose service-role keys or put authorization decisions only in the frontend.
- Run type checks, relevant tests, and a production build after meaningful changes.
- Report changed files, migrations, tests, known gaps, and any measured performance values.
- Use small feature branches and review diffs; never make a broad rewrite without explicit approval.
- Do not claim a test passed if it was not run.

Starter prompt for Antigravity

Use this after creating a clean project folder:

```
Act as the principal architect for a new restaurant management SaaS platform. Build from scratch; do not reuse code, UI, database schemas, or business logic from any previous POS. We will pilot in one café, then support multiple restaurant tenants and branches in one shared PostgreSQL database with strict tenant isolation. Stack: React + TypeScript + Vite, Tailwind CSS, accessible Radix UI primitives, React Router, TanStack Query, Zod/React Hook Form, Supabase Auth/PostgreSQL/Storage, RLS, Vercel. Scope includes POS, KOT, QR menu, inventory/recipes, purchasing, expenses/accounts/financial reporting, OCR with review, staff/cleaning checklists, CRM, and SaaS administration. Mobile-first and performance budgets are required. For now, do not build the whole application. Produce the product specification, architecture diagram, logical data model, RLS/authorization strategy, UI navigation and design-system plan, performance budgets, risks, and phased roadmap. Clearly list assumptions and unresolved decisions. Wait for approval before implementing code.
```

The starter prompt is intentionally shorter than this report. The report is the product baseline; each later prompt should reference the approved requirements and implement only its assigned phase.

18. Appendix: initial logical table inventory

This is an initial planning inventory, not a final SQL schema. Every table needs an explicit tenant ownership classification, authorization policy, lifecycle policy, and indexing strategy.

Area	Initial table names
Tenancy	organizations, branches, organization_memberships, branch_memberships, roles, permissions, role_permissions
Brand/settings	restaurant_settings, brand_assets, custom_domains, operating_hours, tax_settings
Menu/QR	menu_categories, menu_items, item_variants, modifier_groups, modifiers, restaurant_tables, qr_codes
Orders/KOT	orders, order_items, order_item_modifiers, order_events, kot_tickets, kot_items, kitchen_stations, payments, refunds
Inventory	inventory_items, units_of_measure, unit_conversions, recipes, recipe_items, stock_batches, stock_movements, stock_counts, wastage_records
Purchasing	suppliers, purchase_orders, purchase_order_items, goods_receipts, goods_receipt_items, supplier_invoices, supplier_payments
Finance	expenses, expense_categories, financial_transactions, cash_sessions, cash_movements, settlement_records
OCR	documents, document_jobs, ocr_extractions, ocr_line_items, document_reviews, document_postings
Staff	employees, shifts, attendance_events, task_templates, checklist_templates, checklist_runs, checklist_completions, compliance_documents
CRM	customers, customer_consents, feedback, loyalty_accounts, loyalty_transactions, promotions
Platform	subscriptions, plan_entitlements, usage_records, audit_events, idempotency_records, background_jobs, webhook_events

Illustrative route map

Route family	Purpose
/menu/:tenantSlug	Public restaurant-branded QR menu.
/order/:publicOrderRef	Public order status, if enabled with secure reference rules.
/app/overview	Owner/manager overview.
/app/pos	Order-entry and checkout.
/app/kitchen	Kitchen display.
/app/orders	Order search and management.
/app/menu	Menu and modifier configuration.
/app/inventory	Stock, recipes, wastage, and counts.
/app/purchasing	Suppliers, orders, receipts, and invoices.
/app/finance	Expenses, cash, reports, and reconciliation.
/app/documents	OCR upload, review, and approval.
/app/staff	Employees, shifts, duties, and checklists.
/app/customers	CRM, loyalty, and feedback.
/app/settings	Restaurant, branches, users, branding, and configuration.
/platform/*	Restricted SaaS administration.

Final product definition

Build a new, mobile-first restaurant operating system that connects orders, kitchen preparation, inventory, purchasing, finance, staff operations, document automation, and customer relationships. Establish tenant isolation and design quality before expanding functionality. Validate the full workflow in one café, then onboard other restaurants through configuration rather than separate deployments.

End of document • Version 1.0 • Prepared 18 October 2026